Universidad Tecnológica de El Salvador

FACULTAD DE INFORMÁTICA Y CIENCIAS APLICADAS
ESCUELA DE INFORMÁTICA

PROGRAMACIÓN III

DOCENTE:
ING. ELISEO ERNESTO HERNÁNDEZ DIAZ

SECCIÓN: 03

TEMA DE INVESTIGACION:
COMPARACIÓN DEL RENDIMIENTO DE UNA APLICACIÓN WEB Y UNA APLICACIÓN ANDROID AL CONSUMIR UNA API REST PARA UN SISTEMA DE INVENTARIO CON SQL SERVER

EQUIPO N. 01

INTEGRANTES:

No.	Nombre Completo	N° Carnet	Usuario GitHub
1	Diego Alberto Chicas Martinez		Sin participacion
2	José David Grande Lorenzana	25-4832-2022	DavidGrande06
3	Alejandra Carolina Guillen Campos	27-1517-2025	aguillencampos
4	Nelson Eduardo Ramirez Ramirez	25-2610-2024	nelsoneduardoramirezramirez-cell
5	Miguel Angel Rivera Lopez	25-0948-2023	miguelrivera102004-bot
6	René Daniel Ventura Sibrian	27-0417-2025	renev2704

SAN SALVADOR, 16 DE OCTUBRE DE 2026

Estado del arte

Búsqueda y selección de antecedentes

Para identificar investigaciones relacionadas con el rendimiento de aplicaciones que consumen servicios web, se consideran estudios sobre aplicaciones móviles nativas e híbridas, bibliotecas de comunicación mediante REST API y herramientas para medir el tiempo de respuesta y el consumo de recursos. La búsqueda se orienta a trabajos con mediciones cuantitativas que permitan comparar una aplicación web y una aplicación Android bajo condiciones equivalentes.

Las fuentes de consulta incluyen revistas académicas, repositorios científicos y documentación técnica oficial, priorizando trabajos publicados entre 2021 y 2026, sin excluir investigaciones anteriores cuando aportan fundamentos relevantes.

Cadenas de búsqueda propuestas:

Tema	Cadena de búsqueda	Base de datos	Fecha	Resultados	Seleccionados
Rendimiento web vs móvil	("web application" OR "mobile application" OR Android) AND ("REST API" OR "RESTful API") AND ("performance comparison" OR "response time" OR "memory consumption")	IEEE Xplore	5 09 2026	127	6
Android + REST + rendimiento	(Android OR Kotlin) AND ("REST API") AND ("performance" OR "latency" OR "memory") AND (Retrofit OR OkHttp)	ACM Digital Library	5 09 2026	89	4
SQL Server + API REST + rendimiento	("SQL Server" OR "Microsoft SQL Server") AND ("REST API") AND ("CRUD" OR "inventory") AND ("performance evaluation" OR "benchmark")	Scopus / ScienceDirect	5 09 2026	45	3

Los criterios de inclusión son la relación directa con el rendimiento de aplicaciones, la disponibilidad de información sobre el método utilizado y la presencia de métricas o resultados verificables. Se excluyen los trabajos que no describen suficientemente su procedimiento, los documentos duplicados y las fuentes comerciales que no presentan evidencia comprobable.

Matriz comparativa de antecedentes

Autor y año	Objetivo	Método y entorno	Hallazgo principal	Vacío detectado
Kaczmarczyk et al. (2022)	Comparar aplicaciones Android nativas e híbridas que procesan información mediante una REST API en escenarios de sensores BLE/Wi-Fi	Java 8 nativo vs. Ionic híbrido; red de sensores con BLE/Wi-Fi; medición de tiempos de construcción y respuesta	La app nativa construye 2× más rápido; responde 2× más rápido a la REST API, y la ventaja crece con el número de objetos en la respuesta; usa ~20% menos recursos de hardware	No incluye aplicación web; sin SQL Server; no es sistema de inventario; no compara consumo de memoria entre plataformas
Bauer (2025)	Comparar Kotlin Multiplatform, Flutter y nativo iOS/Android en rendimiento	Hardware controlado; peticiones HTTP GET RESTful, acceso a base de datos, renderizado UI, tareas intensivas de CPU	iOS nativo superior; Flutter fuerte en CPU pero ineficiente en memoria; Kotlin Multiplatform equilibrado pero complejo; Android nativo balanceado	Solo compara móvil; sin contraste con aplicación web; sin SQL Server; sin escenario de inventario
Santoso et al. (2025)	Comparar Retrofit, Ktor Client, OkHttp y Volley en la obtención de datos en Android	Peticiones HTTP POST/GET; entre 100 y 10,000 registros; medición de tiempo de respuesta, memoria y datos transmitidos	OkHttp responde más rápido; Retrofit es más eficiente en memoria; Ktor Client transmite menos datos; sin diferencias significativas en la recepción	Solo Android, sin contraste web; sin SQL Server; no es escenario CRUD de inventario
Pratama et al. (2025)	Comparar el rendimiento de las bibliotecas Retrofit y Volley en la obtención de datos mediante REST API	Pruebas con REST API; medición de tiempo de respuesta, uso de red y consumo de memoria	Diferencias medibles según la biblioteca y la configuración; Retrofit más eficiente en memoria, Volley mejor en tiempo de respuesta	Solo Android; sin contraste web; sin SQL Server; un solo volumen de datos

IEEE ICWS (2015)	Caracterizar diferencias de rendimiento entre app nativa y app web al consumir servicios RESTful	Análisis de trazas a nivel HTTP; GET, DELETE, PUT, POST; número de conexiones, tiempo de respuesta, consumo de datos	Las apps web no siempre son peores que las nativas; los patrones de uso difieren según el estilo arquitectónico; se proponen mejoras	El más cercano pero desactualizado (10 años); sin SQL Server; no es inventario; no mide memoria
REST vs GraphQL en móvil (2023)	Comparar REST y GraphQL en calidad de software desde aplicaciones móviles	Experimento computacional; eficiencia de rendimiento; 3 APIs (1 GraphQL, 2 REST)	GraphQL supera a REST en calidad de software desde móvil	Compara arquitecturas de API, no plataformas; solo móvil; sin contraste web; sin SQL Server
Kotlin vs Flutter vs React Native (2025)	Comparar tiempo de respuesta y uso de recursos en llamadas frecuentes a API	Llamadas frecuentes a API; medición de respuesta, CPU, memoria y renderizado	Kotlin nativo es el más rápido y estable; Flutter tiene baja latencia y renderizado fluido; React Native sufre por el puente JS	Compara frameworks móviles; sin aplicación web; sin SQL Server
Xamarin vs nativo (Universidad de Tartu)	Comparar velocidad de arranque y peticiones REST entre Xamarin y desarrollo nativo	Peticiones con arquitectura REST; inserción/lectura masiva en SQLite (10,000 registros)	La app nativa arranca ~2× más rápido; las peticiones REST son ~2× más rápidas; Xamarin usa ~20× más memoria que nativo	Compara frameworks, no web vs móvil; sin SQL Server; sin CRUD de inventario

Párrafos de síntesis por antecedente

Kaczmarczyk et al. (2022): Este estudio demuestra que las aplicaciones nativas superan a las híbridas en velocidad de construcción y respuesta a REST APIs, con una ventaja que se amplifica conforme crece el volumen de datos. Su aporte principal es evidenciar que la plataforma de ejecución influye directamente en el rendimiento al consumir servicios REST. Sin embargo, al no incluir una aplicación web como contraste, no permite determinar si esas diferencias son exclusivas de Android o comunes a cualquier cliente. Este vacío refuerza la necesidad de comparar web y Android bajo condiciones equivalentes.

Bauer (2025): Aporta una comparación controlada entre Kotlin Multiplatform, Flutter y desarrollo nativo en iOS y Android, midiendo peticiones HTTP, acceso a base de datos y renderizado. Sus resultados muestran que el rendimiento óptimo depende del tipo de tarea y que la memoria es un factor crítico diferenciador entre frameworks. No obstante, el estudio se limita al ecosistema móvil y omite cualquier contraste con aplicaciones web, lo que deja sin responder si una aplicación web podría igualar o superar el rendimiento observado.

Santoso et al. (2025): Compara cuatro bibliotecas de red en Android (Retrofit, Ktor Client, OkHttp y Volley) bajo volúmenes crecientes de datos, midiendo tiempo de respuesta, memoria y datos transmitidos. Su hallazgo principal es que no existe una biblioteca óptima para todas las métricas: OkHttp gana en velocidad, Retrofit en memoria y Ktor en eficiencia de red. Este antecedente es metodológicamente cercano al presente estudio por su diseño con volúmenes variables y múltiples métricas, pero se restringe a Android y no incorpora una aplicación web ni un backend SQL Server.

Pratama et al. (2025): Compara Retrofit y Volley en la obtención de datos mediante REST API, midiendo tiempo de respuesta, uso de red y consumo de memoria. Sus resultados

confirman que la elección de biblioteca afecta el rendimiento, pero al centrarse en Android y en un único volumen de datos, no responde la pregunta sobre diferencias entre aplicación web y Android ni sobre cómo varía la brecha con el volumen de registros.

IEEE ICWS (2015): Es el antecedente más directamente relacionado con esta investigación, pues compara aplicaciones nativas y aplicaciones web al consumir servicios RESTful mediante análisis de trazas HTTP. Su hallazgo más relevante es que las aplicaciones web no siempre son inferiores a las nativas, lo que contradice la intuición común y justifica la necesidad de mediciones empíricas. No obstante, su antigüedad (10 años), la ausencia de SQL Server, la falta de un escenario de inventario y la omisión del consumo de memoria limitan su aplicabilidad directa al presente estudio.

REST vs GraphQL en móvil (2023): Compara dos arquitecturas de API en aplicaciones móviles, encontrando que GraphQL supera a REST en calidad de software medida desde el cliente. Su aporte es demostrar que la arquitectura de la API afecta el rendimiento del cliente, variable que este estudio mantiene constante (REST) para aislar el efecto de la plataforma. Al centrarse en arquitecturas y no en plataformas, deja abierto el contraste web vs Android.

Kotlin vs Flutter vs React Native (2025): Evalúa el rendimiento de tres enfoques de desarrollo móvil en llamadas frecuentes a API, concluyendo que Kotlin nativo es el más rápido y estable, Flutter destaca en renderizado y React Native sufre por el puente JavaScript. Este antecedente confirma que la tecnología del cliente determina el rendimiento, pero al comparar solo frameworks móviles, no aporta evidencia sobre cómo se comportaría una aplicación web bajo las mismas condiciones.

Xamarin vs nativo (Universidad de Tartu): Compara el rendimiento de Xamarin frente al desarrollo nativo en arranque y peticiones REST con operaciones masivas sobre SQLite

(10,000 registros). Sus resultados muestran que el desarrollo nativo es aproximadamente 2× más rápido en arranque y peticiones, y usa hasta 20× menos memoria. Este estudio aporta evidencia sobre el impacto de la tecnología en el rendimiento y la memoria, pero al comparar frameworks y no plataformas (web vs Android), y al usar SQLite en lugar de SQL Server, no cubre el vacío que esta investigación aborda.

Marco teórico

Eficiencia de desempeño según ISO/IEC 25010

La norma ISO/IEC 25010 define la eficiencia de desempeño como la capacidad de un producto de software para proporcionar un rendimiento apropiado, en relación con la cantidad de recursos utilizados bajo condiciones establecidas. Esta característica se descompone en tres subcaracterísticas medibles: comportamiento temporal (tiempos de respuesta y procesamiento), utilización de recursos (cantidad y tipos de recursos consumidos) y capacidad (límites máximos de parámetros del sistema) .

Un estudio sistemático reciente sobre eficiencia de desempeño en ISO/IEC 25010, que revisó 38 estudios primarios entre 2014 y 2024, identificó que el comportamiento temporal es la subcaracterística más investigada (48% de los estudios), seguida de la utilización de recursos (36%) y la capacidad (16%) . Este mismo estudio catalogó 68 métricas específicas para evaluar estas subcaracterísticas, lo que proporciona una base objetiva para medir y comparar el rendimiento del software.

ISO/IEC 25010 resulta el estándar adecuado para esta investigación porque proporciona un vocabulario preciso y operacionalizable para definir las métricas de comparación. En lugar de hablar genéricamente de “rapidez” o “ligereza”, la norma permite especificar que se medirá el comportamiento temporal (tiempo de respuesta en milisegundos) y la utilización de recursos

(consumo de memoria en megabytes) bajo condiciones controladas y reproducibles. Esta delimitación conceptual es requisito indispensable para que los resultados de la comparación entre aplicación web y aplicación Android sean interpretables y comparables con la literatura existente.

Arquitecturas de consumo de APIs REST

REST (Representational State Transfer) es un estilo arquitectónico para sistemas distribuidos, no una tecnología ni un toolkit específico. Su implementación se rige por una serie de restricciones de diseño que, cuando se cumplen en su totalidad, permiten calificar un servicio como “RESTful” . Las restricciones más relevantes para el consumo desde aplicaciones cliente son:

Cliente-servidor: Separa las preocupaciones de interfaz de usuario (cliente) de las de almacenamiento y procesamiento de datos (servidor), permitiendo que cada componente evolucione independientemente.

Sin estado (stateless): El servidor no mantiene información de sesión entre llamadas. Cada petición debe contener toda la información necesaria para ser procesada, lo que simplifica la escalabilidad del servidor, pero traslada al cliente la responsabilidad de gestionar el estado .

Interfaz uniforme: Los recursos se identifican mediante URIs y se manipulan con un conjunto fijo de métodos HTTP (GET, POST, PUT, DELETE). Esta uniformidad simplifica el desacoplamiento arquitectónico.

Cacheable: Las respuestas deben indicar explícitamente si son cacheables o no, lo que permite reducir interacciones y mejorar el rendimiento percibido.

La diferencia fundamental entre consumir una API REST desde una **aplicación web** (HTML/CSS/JavaScript en navegador) y desde una **aplicación Android** (Kotlin/Retrofit) reside en el **modelo de ejecución y gestión de recursos** de cada plataforma:

En la **aplicación web**, el navegador gestiona el ciclo de vida de las peticiones mediante fetch o XMLHttpRequest, aplica políticas de seguridad como CORS (Cross-Origin Resource Sharing), y delega la gestión de memoria al motor JavaScript del navegador. El costo de serialización/deserialización JSON ocurre en el hilo principal o en Web Workers, y la memoria está sujeta a las políticas de recolección de basura del navegador.

En la **aplicación Android**, el framework proporciona bibliotecas especializadas como Retrofit/OkHttp que operan sobre el modelo de concurrencia de Android (hilos, corrutinas). La serialización JSON se realiza en el hilo de fondo, y la memoria utilizada proviene del heap gestionado por ART (Android Runtime), sujeto a las restricciones específicas de memoria por aplicación que impone el sistema operativo.

Un estudio empírico que comparó aplicaciones nativas y aplicaciones web consumiendo los mismos servicios RESTful encontró que **las aplicaciones web no siempre presentan peor rendimiento que las nativas**. Los patrones de uso difieren según el estilo arquitectónico de cada plataforma, y variables como el número de conexiones de red, el tiempo de respuesta y el consumo de datos muestran diferencias que dependen del contexto.

Métricas de rendimiento en aplicaciones

Las métricas de rendimiento cuantifican aspectos observables del comportamiento del software. Para esta investigación, las métricas centrales derivadas de ISO/IEC 25010 son:

Tiempo de respuesta: Intervalo transcurrido entre el envío de una petición y la recepción completa de la respuesta. Esta métrica corresponde a la subcaracterística de

comportamiento temporal. En aplicaciones web puede capturarse mediante performance.now() en JavaScript o la API de Performance del navegador. En Android, se registra mediante instrumentación con System.nanoTime() antes y después de la llamada al servicio.

Consumo de memoria: Cantidad de memoria retenida durante la ejecución de una operación. Esta métrica corresponde a la subcaracterística de utilización de recursos. La medición debe realizarse con instrumentos específicos de cada plataforma, ya que los modelos de memoria son fundamentalmente distintos.

Instrumentos de medición

Para aplicación web:

- **Chrome DevTools (Performance panel):** permite registrar y analizar métricas como LCP (Largest Contentful Paint), CLS (Cumulative Layout Shift) e INP (Interaction to Next Paint), además de proporcionar una línea de tiempo detallada de actividad de red, renderizado y ejecución de JavaScript. La API performance.memory permite obtener estimaciones del heap JavaScript utilizado.

Para aplicación Android:

- **Android Profiler (Memory Profiler):** componente de Android Studio que muestra el uso de memoria en tiempo real, permite capturar heap dumps, forzar recolección de basura y rastrear asignaciones. Las categorías de memoria reportadas incluyen Java/Kotlin, Native, Graphics, Stack, Code y Other.

Para backend y análisis complementario:

- **JMH (Java Microbenchmark Harness):** estándar de facto para microbenchmarks en JVM. Permite controlar iteraciones de calentamiento, mediciones y evitar sesgos del compilador JIT.

- **Apache JMeter:** herramienta para pruebas de carga que simula usuarios concurrentes y mide throughput, tiempo de respuesta y tasas de error sobre endpoints HTTP .

La selección de estos instrumentos está fundamentada en su capacidad para producir mediciones **comparables con las reportadas en la literatura** y en su adecuación a las subcaracterísticas de ISO/IEC 25010 que esta investigación operacionaliza.

Sistemas de inventario y operaciones CRUD

Un sistema de inventario es un tipo de sistema de información diseñado para gestionar el flujo de productos, existencias y movimientos dentro de una organización. Las operaciones fundamentales sobre los recursos de inventario (productos, categorías, movimientos de stock) corresponden al paradigma **CRUD** (Create, Read, Update, Delete), que en el contexto de APIs REST se mapea directamente a los métodos HTTP POST, GET, PUT/PATCH y DELETE .

El sistema de inventario es un caso representativo para esta investigación por tres razones:

Operaciones repetitivas: Las consultas de productos y actualizaciones de existencias son operaciones de alta frecuencia en entornos empresariales, lo que las hace susceptibles a diferencias de rendimiento que se acumulan con el uso.

Volúmenes crecientes de datos: Un catálogo de inventario puede variar desde cientos hasta cientos de miles de registros. Esta variabilidad permite estudiar cómo el rendimiento de cada plataforma cliente se degrada (o no) a medida que aumenta el volumen de información transferida.

Payloads de tamaño variable: Las respuestas de un endpoint de listado de productos escalan linealmente con el número de registros solicitados, lo que expone de manera directa las diferencias en los costos de serialización, transferencia y renderizado entre plataformas.

Esta categoría justifica el dominio de aplicación elegido, ya que el inventario proporciona un escenario realista donde las decisiones sobre plataforma cliente (web vs Android) tienen implicaciones prácticas para desarrolladores y organizaciones que implementan sistemas multiplataforma.

Operacionalización de variables

Variable	Definición operacional	Unidad	Instrumento
Tiempo de respuesta web	ms entre envío de petición fetch y recepción del último byte en Chrome	ms	Chrome DevTools / Performance API (performance.now())
Tiempo de respuesta Android	ms entre envío con Retrofit y callback onResponse	ms	Android Profiler + logs con System.nanoTime()
Consumo de memoria web	MB retenidos en heap JS durante la operación	MB	performance.memory (Chrome)
Consumo de memoria Android	MB retenidos en heap Java/Kotlin durante la operación	MB	Android Profiler (Memory Profiler)
Volumen de registros	Número de filas en tabla productos/existencias	entero	Script SQL de seeding
Tipo de operación	CRUD: Create, Read, Update, Delete	categórico	Endpoint invocado

Diseño metodológico

Tipo y enfoque de investigación

Esta investigación adopta un diseño de **benchmarking comparativo** con enfoque cuantitativo y alcance correlacional-explicativo. Según la clasificación presentada en la guía, el benchmarking comparativo es el diseño adecuado cuando se busca medir el desempeño de herramientas o configuraciones bajo una carga de trabajo definida, con calentamiento previo y series de mediciones repetidas.

El estudio compara dos tratamientos técnicos —una aplicación web y una aplicación Android— que consumen los mismos endpoints de una API REST. No se manipulan variables de manera arbitraria, sino que se observa el comportamiento de cada plataforma bajo condiciones controladas y equivalentes. Por ello, el diseño también puede caracterizarse como **cuasi-experimental** con dos grupos de tratamiento (web y Android) y una variable de bloqueo (volumen de registros).

El enfoque es **cuantitativo** porque todas las variables respuesta se expresan en unidades numéricas (milisegundos y megabytes) y se analizan mediante estadística descriptiva e inferencial. El alcance es **comparativo** porque el objetivo no es describir cada plataforma por separado, sino determinar si existen diferencias significativas entre ellas y en qué condiciones se manifiestan.

Diseño del experimento

El experimento se estructura en torno a tres factores, una variable respuesta múltiple, un número definido de repeticiones y un conjunto de condiciones controladas.

Factores y niveles

Factor	Tipo	Niveles
Factor A: Plataforma de la aplicación	Categórico	Aplicación web (Chrome 128) / Aplicación Android (Kotlin 2.0 + Retrofit 2.11)
Factor B: Volumen de registros	Categórico ordinal	1.000 / 10.000 / 100.000 registros
Factor C: Tipo de operación CRUD	Categórico	Create (POST) / Read (GET) / Update (PUT) / Delete (DELETE)

La combinación completa de factores produce 2 × 3 × 4 = 24 celdas experimentales. Cada celda representa una condición única de prueba (por ejemplo: aplicación web, 10.000 registros, operación Read).

Variables respuesta

Variable	Definición operacional	Unidad	Instrumento
Tiempo de respuesta web	ms entre envío de petición fetch y recepción del último byte	ms	performance.now() + Chrome DevTools
Tiempo de respuesta Android	ms entre envío con Retrofit y callback onResponse	ms	System.nanoTime() + Android Profiler
Consumo de memoria web	Memoria máxima retenida en heap JavaScript	MB	performance.memory
Consumo de memoria Android	Memoria máxima retenida en heap Java/Kotlin	MB	Android Profiler (Memory Profiler)

Repeticiones y calentamiento

Se realizarán 10 repeticiones por celda, precedidas por 5 ejecuciones de calentamiento no registradas. Total: 240 ejecuciones registradas + 120 de calentamiento = 360 ejecuciones.

Condiciones controladas

- **Misma máquina servidor:** Intel Core i5-11400, 16 GB RAM, Windows 11.
- **Misma red local:** Ambas aplicaciones en la misma LAN, sin Internet.
- **Sin otras cargas:** Servidor sin procesos adicionales; dispositivo Android reiniciado antes de cada serie.
- **Caché de SQL Server reiniciada** entre series.
- **Caché del navegador deshabilitada** con Cache-Control: no-cache.
- **Datos sintéticos idénticos** generados con script SQL reproducible.

Orden de ejecución

El orden de las 24 celdas se aleatoriza con semilla fija (seed = 42). Dentro de cada celda, las 10 repeticiones se ejecutan consecutivamente con pausa de 2 segundos entre ellas.

Población, muestra o corpus

Esta investigación no trabaja con personas, sino con artefactos de software y datos sintéticos.

Corpus de datos

Volumen	Descripción	Regla de generación
1.000 registros	Catálogo pequeño	Script SQL con semilla fija

10.000 registros	Catálogo mediano	Script SQL con semilla fija
100.000 registros	Catálogo grande	Script SQL con semilla fija

Justificación: los tres volúmenes cubren tres órdenes de magnitud y permiten observar si la diferencia entre plataformas se mantiene, se amplifica o se invierte con el crecimiento. Son consistentes con los volúmenes usados en la literatura revisada.

Unidades de análisis

Cada **ejecución individual** de una operación CRUD sobre una plataforma y volumen determinados constituye una unidad de análisis, produciendo dos mediciones: tiempo de respuesta (ms) y consumo de memoria (MB).

Plan de análisis

El plan se define **antes** de la recolección de datos para evitar sesgos de confirmación.

Etapa 1 — Estadística descriptiva: media, mediana, desviación estándar, coeficiente de variación y diagramas de caja por celda.

Etapa 2 — Verificación de supuestos: Shapiro-Wilk para normalidad ($p > 0.05$) y Levene para homogeneidad de varianzas ($p > 0.05$).

Etapa 3 — Pruebas de comparación:

Comparación	Paramétrica	No paramétrica
Web vs Android (por celda)	t de Student	Mann-Whitney U
Tres volúmenes (por plataforma)	ANOVA	Kruskal-Wallis
Interacción plataforma × volumen	ANOVA de dos factores	—

Nivel de significancia: $\alpha = 0.05$, con corrección de Bonferroni para comparaciones múltiples.

Etapa 4 — Tamaño del efecto: d de Cohen (0.2 pequeño, 0.5 medio, 0.8 grande) y η^2 (0.01 pequeño, 0.06 medio, 0.14 grande).

Etapa 5 — Intervalos de confianza al 95% para medias y diferencias.

Etapa 6 — Análisis de umbrales: gráfico de diferencia porcentual vs volumen; punto de inflexión donde supera el 20% (umbral práctico a priori).

Herramientas: Python 3.12 (pandas, scipy.stats, matplotlib).

Amenazas a la validez

Tipo	Riesgo	Mitigación
Interna	Procesos en segundo plano en Android	Reinicio del dispositivo; cierre de apps; modo avión para datos móviles
Interna	Caché del navegador acelera peticiones web	Caché deshabilitada; headers no-cache; perfil limpio
Interna	Caché de SQL Server beneficia consultas repetidas	Reinicio de caché entre series
Externa	Resultados solo aplican a SQL Server 2022	Declarar alcance; recomendar replicación en PostgreSQL/MySQL
Externa	Resultados solo aplican al hardware usado	Documentar hardware; declarar que valores absolutos pueden variar
De constructo	performance.memory no comparable con Android Profiler	Reportar por separado; no mezclar en un mismo gráfico
De constructo	Tiempo percibido ≠ tiempo medido	Medir red y tiempo total; declarar qué se mide
De conclusión	10 repeticiones pueden ser insuficientes	Reportar dispersión y CV; usar pruebas no paramétricas; calcular tamaño del efecto
De conclusión	Correlación no implica causalidad	Solo describir diferencias observadas bajo condiciones controladas

Consideraciones éticas

Esta investigación no involucra seres humanos, por lo que no requiere consentimiento informado ni aprobación de comité de ética.

Datos: todos los datos son **sintéticos** y generados por script. No se utilizan datos reales ni información sensible.

Licencias de software: todas las herramientas son de acceso libre o gratuito para desarrollo (SQL Server Developer, OpenJDK 21, Spring Boot Apache 2.0, Kotlin Apache 2.0, Retrofit/OkHttp Apache 2.0, Chrome gratuito, Python PSF/BSD).

Declaración de uso de asistentes de IA: se utilizó un asistente de IA como apoyo para revisión de redacción, generación del script de seeding y verificación de sintaxis de cadenas de búsqueda. Todas las decisiones metodológicas, selección de métricas, diseño experimental y análisis de resultados fueron realizados por el autor. El uso se declara en esta sección y en el README.md del repositorio.

Reproducibilidad: el repositorio incluye scripts de generación de datos, configuraciones de entorno y documentación de decisiones para replicar el estudio.

Referencias bibliográficas

Android Developers. (s. f.). *Overview of memory management*. Google. https://developer.android.com/topic/performance/memory-overview

Bauer, S. (2025). *Kotlin Multiplatform, Flutter and Native iOS/Android: A performance comparison* [Tesis de maestría, FH Oberösterreich]. https://pure.fh-ooe.at/en/studentTheses/kotlin-multiplatform-flutter-and-native-iosandroida-performance-c/

Chrome for Developers. (s. f.). *Network features reference*. Google. https://developer.chrome.com/docs/devtools/network/reference/

International Organization for Standardization. (2011). *ISO/IEC 25010:2011: Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—System and software quality models*. https://www.iso.org/standard/35733.html

Kaczmarczyk, A., Zając, P., & Zabierowski, W. (2022). Performance comparison of native and hybrid Android mobile applications based on sensor data-driven applications based on Bluetooth Low Energy (BLE) and Wi-Fi communication architecture. *Energies*, *15*(13), 4574. https://doi.org/10.3390/en15134574

Liu, Y., Liu, X., Ma, Y., Liu, Y., Zheng, Z., Huang, G., & Blake, M. B. (2015). Characterizing RESTful Web services usage on smartphones: A tale of native apps and web apps. En *2015 IEEE International Conference on Web Services (ICWS)* (pp. 337–344). IEEE. https://doi.org/10.1109/ICWS.2015.53

Pratama, B. M. H., Hanggara, B. T., & Putra, W. H. N. (2025). Analisis perbandingan performa networking library Retrofit dan Volley dalam pengambilan data menggunakan REST

API. *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer*, *9*(4). https://j-ptiik.ub.ac.id/index.php/j-ptiik/article/view/14725

Santoso, [iniciales], et al. (2025). *Performance comparison of Retrofit, Ktor Client, OkHttp and Volley in Android data fetching*. [Fuente del repositorio Garuda].

Universidad Técnica del Norte. (2023). *GraphQL o REST para aplicaciones móviles: Efectos en la calidad del software*. Repositorio Digital UTN. https://repositorio.utn.edu.ec/handle/123456789/19674

Universidad de Tartu. (s. f.). *Xamarini raamistikku platvormipõhiste tehnoloogiatega* [Comparación del framework Xamarin con tecnologías nativas]. Digikogu TalTech.